

TDD Leitfaden:

Implementierung der MyList Klasse

```
package org.example;

import org.junit.jupiter.api.Test;
import java.util.List;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class MyListTest {

    private final MyList<String> myList = new MyList<>(List.of("Ada",
"Grace", "Adele", "Linus"));

    // contains mit Objekt

    @Test
    void containsTrue() {
        // Testet den "Happy Path": Das Element ist exakt so in der Liste.
        assertTrue(myList.contains("Grace"));
    }

    @Test
    void containsFalse() {
        // Testet ein nicht vorhandenes Element. "Alan" fehlt in der Liste.
        assertFalse(myList.contains("Alan"));
    }

    @Test
    void containsFalseCaseSensitive() {
        // Testet die Grenzen der Standard-contains-Methode.
        // Da Strings in Java case-sensitive sind, ist "grace" ungleich
"Grace".
        // Hier zeigt sich der Bedarf für flexiblere Methoden (siehe
nächste Tests).
        assertFalse(myList.contains("grace"));
    }

    // contains mit Lambda -----
    -----

    @Test
    void containsLambdaTrue() {
        // Hier übergeben wir Logik statt Daten.
        // Der Lambda-Ausdruck 'element -> element.equals("Grace")'
implementiert das Predicate-Interface.
        // Das Verhalten ist identisch zum normalen contains, aber wir
definieren es von außen.
        assertTrue(myList.contains(element -> element.equals("Grace")));
    }

    @Test
    void containsLambdaTrueCaseInsensitive() {
        // Der entscheidende Vorteil von Higher-Order Functions:
        // Wir ändern die Vergleichslogik zu 'equalsIgnoreCase', OHNE die
```

```

MyList-Klasse anpassen zu müssen.
    // Dies entspricht dem Open-Closed-Principle (offen für
Erweiterung, geschlossen für Modifikation).
    assertTrue(myList.contains(element ->
element.equalsIgnoreCase("grace")));
}

@Test
void containsLambdaFalse() {
    // Auch mit Lambda: Wenn kein Element "alan" (exakt) ist, kommt
false zurück.
    assertFalse(myList.contains(element -> element.equals("alan")));
}

// contains mit Lambda, aber anderer Typ als String -----
-----

@Test
void containsOtherTypeTrue() {
    // Beweis für die Generics: Wir nutzen MyList hier mit Integer
statt String.
    // 'var' (Type Inference) spart uns das Schreiben von
MyList<Integer>.
    var myIntegerList = new MyList<>(List.of(1, 2, 3));

    // Lambda prüft auf numerische Gleichheit.
    assertTrue(myIntegerList.contains(element -> element.equals(3)));
}

@Test
void containsOtherTypeFalse() {
    var myIntegerList = new MyList<>(List.of(1, 2, 3));
    // 4 ist nicht in der Liste -> false.
    assertFalse(myIntegerList.contains(element -> element.equals(4)));
}
}

```

In diesem Guide entwickeln wir die Klasse **MyList** basierend auf der bereitgestellten Testklasse **MyListTest**. Wir folgen dem Zyklus:

-  **RED:** Test schreiben/ausführen (er schlägt fehl oder kompiliert nicht).
-  **GREEN:** Die absolut minimale Änderung machen, damit der Test besteht.

Schritt 1: Die Basis schaffen (Compilation Fixes)

Wenn wir `MyListTest` jetzt ausführen, kompiliert er nicht, weil die Klasse `MyList` gar nicht existiert.

Status: ● Kompiliert nicht.

Aktion: Wir erstellen die Klasse und den Konstruktor.

Hier stehen wir vor der Wahl: Schreiben wir die Klasse spezifisch für Strings (da der erste Test `MyList<String>` verwendet) oder generisch als `MyList<T>`?

Ein Blick in den Test zeigt zwei verschiedene Anwendungsfälle:

1. `new MyList<>(List.of("Ada", "Grace", ...))` -> Hier wird `MyList<String>` benötigt.
2. `new MyList<>(List.of(1, 2, 3))` (`in containsOtherTypeTrue`) -> Hier wird `MyList<Integer>` benötigt.

Würden wir die Klasse `MyList` so schreiben, dass sie fest String erwartet (um den ersten Test zu erfüllen), würde der Testfall mit den Integeren später nicht kompilieren. Um Code-Duplizierung zu vermeiden und **beide** Varianten (`<String>` und `<Integer>`) mit einer Klasse abzudecken, **müssen** wir Generics verwenden.

```
package org.example;

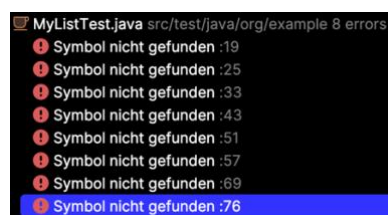
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.function.Predicate;

// <T> ist ein Platzhalter für den Typ (z.B. String oder Integer)
public class MyList<T> {

    private final List<T> elements;

    // Der Test übergibt List.of(...), was eine Collection ist.
    public MyList(Collection<T> elements) {
        // T sorgt dafür, dass wenn MyList<String> erstellt wird,
        // hier nur Strings rein dürfen. Bei MyList<Integer> nur Integer.
        this.elements = new ArrayList<>(elements);
    }
}
```

Jetzt kompiliert der Konstruktor-Aufruf im Test für beide Fälle (String und Integer), aber die Methodenaufrufe `contains` scheitern noch.



```
MyListTest.java src/test/java/org/example 8 errors
❗ Symbol nicht gefunden :19
❗ Symbol nicht gefunden :25
❗ Symbol nicht gefunden :33
❗ Symbol nicht gefunden :43
❗ Symbol nicht gefunden :51
❗ Symbol nicht gefunden :57
❗ Symbol nicht gefunden :69
❗ Symbol nicht gefunden :76
```

Schritt 2: Die einfache `contains` Methode

Der Test `containsTrue`, `containsFalse` und `containsFalseCaseSensitive` ruft `contains("String")` auf.

Status:  Kompiliert nicht (Methode fehlt).

Aktion: Wir implementieren die Methode `contains`, die ein Objekt vom Typ `T` entgegennimmt. Wir delegieren einfach an die interne Liste.

```
// ... innerhalb der Klasse MyList

// Wir verwenden hier 'T element' statt 'String element'.
// Wäre es 'String', würde der Integer-Test (contains(3)) fehlschlagen.
public boolean contains(T element) {
    return elements.contains(element); // Delegation an die Java-Liste
}
```

Erklärung zur Delegation (`elements.contains`):

Du wunderst dich vielleicht, warum wir hier `contains` aufrufen, obwohl wir gerade in der Methode `contains` sind.

- Das ist **keine** Rekursion.
- Wir rufen `elements.contains(...)` auf. Das `elements`-Objekt ist eine normale Java `ArrayList`.
- Die Java-Standardbibliothek (`java.util.List`) hat bereits eine fertig programmierte und optimierte Logik, um zu prüfen, ob ein Element da ist.
- Anstatt selbst eine Schleife zu schreiben, **delegieren** (leiten) wir die Arbeit einfach an den "Profi" (die interne Liste) weiter. Wir nutzen also vorhandene Funktionalität wieder.

Testlauf:

- `containsTrue`:  Pass
- `containsFalse`:  Pass
- `containsFalseCaseSensitive`:  Pass
- `containsLambda...`:  Fail (Kompiliert noch nicht)

Status:  Teilweise Grün (für Objekte).

✓ MyListTest (org.example)	8 ms
✓ containsTrue()	7 ms
✓ containsFalse()	1 ms
✓ containsFalseCaseSensitive()	

Schritt 3: `contains` mit Lambda (Prädikat)

Die Tests `containsLambdaTrue` etc. rufen `contains` mit einem Lambda-Ausdruck auf (z.B. `element -> element.equals("Grace")`). Java interpretiert diesen Lambda-Ausdruck als `Predicate<T>`.

Da unsere bisherige Methode `contains(T element)` ein Objekt erwartet, passt der Lambda-Ausdruck nicht hinein. Wir müssen die Methode **überladen** (Overloading).

Status:  Kompiliert nicht für Lambda-Tests.

```
MyListTest.java src/test/java/org/example 5 errors
1 Inkompatible Typen: java.lang.String ist keine Funktionsschnittstelle :43
2 Inkompatible Typen: java.lang.String ist keine Funktionsschnittstelle :51
3 Inkompatible Typen: java.lang.String ist keine Funktionsschnittstelle :57
4 Inkompatible Typen: java.lang.Integer ist keine Funktionsschnittstelle :69
5 Inkompatible Typen: java.lang.Integer ist keine Funktionsschnittstelle :76
```

Aktion: Wir fügen eine zweite `contains`-Methode hinzu, die ein `Predicate<T>` akzeptiert.

```
// ... innerhalb der Klasse MyList

public boolean contains(Predicate<T> criteria) {
    for (T element : elements) {
        // Wir fragen das Prädikat: "Passt dieses Element?"
        if (criteria.test(element)) {
            return true;
        }
    }
    return false;
}
```

Exkurs: Was ist `Predicate` und warum nutzen wir es?

1. Was ist Predicate?

Das `Predicate<T>` ist ein funktionales Interface aus `java.util.function`. Es ist extrem simpel aufgebaut. Stell es dir vereinfacht so vor:

```
public interface Predicate<T> {
    boolean test(T t); // Nimm etwas (T) und sag mir: Ja (true) oder Nein (false).
}
```

Wenn du im Test schreibst: `element -> element.equals("Grace")`, erstellt Java im Hintergrund eine Instanz dieses Interfaces und packt deinen Code in die Methode `test`.

2. Warum kein eigenes Interface?

Wir hätten auch unser eigenes Interface schreiben können, z.B.:

```
// Möglich, aber nicht empfohlen:  
interface MySearchCriteria<T> {  
    boolean doesItMatch(T item);  
}
```

Wir nutzen jedoch aus drei Gründen das Standard-`Predicate`:

1. **Wiedererkennungswert (Der "Standard"-Effekt):** Jeder Java-Entwickler kennt `Predicate`. Wenn jemand deinen Code liest, weiß er sofort: "Aha, hier wird eine Bedingung geprüft." Ein eigenes Interface (`MySearchCriteria`) müsste er erst öffnen und verstehen.
2. **Kompatibilität:** Java's Standard-Bibliotheken (z.B. Streams API, `filter`) nutzen alle `Predicate`. Wenn du das Standard-Interface nutzt, kannst du deine Methoden leicht mit anderen Java-Tools kombinieren.
3. **Zusatzfunktionen:** Das `Predicate`-Interface schenkt dir mächtige Methoden dazu, wie `.and()`, `.or()` und `.negate()`. Du könntest später schreiben: `myList.contains(bedingung1.and(bedingung2))`. Bei einem eigenen Interface müsstest du diese Logik selbst programmieren.

Schritt 4: Alles zusammenfügen & Finaler Test

Jetzt haben wir alle Anforderungen erfüllt. Wir führen alle Tests in `MyListTest` aus.

Gesamter Code:

```
package org.example;

import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.function.Predicate;

class MyList<T> {

    private final List<T> elements;

    // Konstruktor
    MyList(Collection<T> elements) {
        this.elements = new ArrayList<>(elements);
    }

    // Implementierung für Schritt 2: Direktes Objekt
    boolean contains(T element) {
        return elements.contains(element);
    }

    // Implementierung für Schritt 3: Lambda / Predicate
    boolean contains(Predicate<T> criteria) {
        for (T element : elements) {
            if (criteria.test(element)) {
                return true;
            }
        }
        return false;
    }
}
```

Testlauf:

- Alle Tests (String-Vergleiche, Case-Insensitive, Integer-Typen) sollten nun bestehen.

Status:  Alles Grün.

✓ MyListTest (org.example)	8 ms
✓ containsTrue()	8 ms
✓ containsFalse()	
✓ containsOtherTypeTrue()	
✓ containsOtherTypeFalse()	
✓ containsLambdaTrueCaseInsensitive()	
✓ containsLambdaTrue()	
✓ containsFalseCaseSensitive()	
✓ containsLambdaFalse()	

Zusammenfassung der TDD-Entscheidungen

1. Warum **Generics <T>** statt reiner String-Implementierung?

Der Test forderte explizit zwei Szenarien:

- `new MyList<>(List.of("A", "B"))` (String)
- `new MyList<>(List.of(1, 2))` (Integer)

Hätten wir die Klasse `MyList` fest auf String programmiert (um den ersten Test zu bestehen), wäre der Integer-Test sofort fehlgeschlagen (Kompilierfehler). Durch die Verwendung von `<T>` (Generics) schreiben wir den Code einmal, und er funktioniert für `MyList<String>`, `MyList<Integer>` und jeden anderen Typ.

2. **Collection<T> im Konstruktor:** `List.of` gibt eine Liste zurück, aber `Collection` ist der allgemeinere Typ, was die Klasse flexibler macht. `new ArrayList<>(...)` wird verwendet, um eine modifizierbare Kopie zu erstellen (obwohl im Test keine Modifikation stattfindet, ist das "Best Practice").
3. **Overloading & Predicate:** Der Schlüssel zum Bestehen der Lambda-Tests war zu erkennen, dass `contains` überladen werden muss. Einmal für den direkten Vergleich (`contains(T)`) und einmal für die funktionale Prüfung (`contains(Predicate<T>)`). Wir nutzen `Predicate`, um Standardkonformität und Flexibilität zu gewährleisten.